

Министерство образования и науки РФ

Федеральное государственное автономное образовательное учебное
учреждение высшего образования “Санкт-Петербургский
национальный исследовательский университет ИТМО”

Факультет прикладной информатики

Лабораторная работа №5

По дисциплине: Проектирование и реализация баз данных

Выполнил:

Ситдилов Рафаэль Ильдусович

Группа: К3240

Проверила:

Говорова Марина Михайловна

Г. Санкт-Петербург, 2026г.

Задание:

Вариант 14. БД «Служба заказа такси»

Описание предметной области: Система должна фиксировать все вызовы такси и распределять их между водителями.

Каждому водителю ежедневно начисляется заработная плата в зависимости от количества вызовов и их тарифа (50% от заработанной им суммы). Автомобили могут быть собственностью компании или таксиста.

Заказ принимает дежурный администратор и передает его водителю. В заказе фиксируется тип оплаты – наличными или онлайн. Если заказ оплачивается онлайн, то в системе хранится привязка к карте, с которой была совершена оплата.

В системе необходимо хранить график работы водителей.

Ежедневно действуют базовые тарифы на тип предоставляемых авто, но в зависимости от времени суток и ситуации на дорогах, цена может корректироваться.

БД должна содержать следующий минимальный набор сведений: Код сотрудника. ФИО сотрудника. Адрес сотрудника. № телефона сотрудника. Паспортные данные сотрудника. Должность сотрудника. Категория сотрудника. Наименование модели и марки автомобиля. Технические характеристики. Стран-производитель. Стоимость. Код тарифа. Наименование тарифа. Цена за километр. Код автомобиля. Госномер автомобиля. Год выпуска. Пробег. Дата последнего ТО. Дата вызова. Время посадки пассажира. Время высадки пассажира. Номер телефона пассажира. Откуда. Куда. Расстояние. Штраф за время ожидания (в минутах). Оплата (онлайн (при заказе) или наличными). Рекламация клиента на вызов.

Отчёт по Лабораторным работам 3 и 4 размещён по ссылкам:

 ЛР3 В14 467478 Ситдигов Р. И. К3240

 ЛР4 В14 467478 Ситдигов Р. И. К3240

Создание процедур

1. Процедура добавления клиента

Была создана процедура `taxi.add_client`, предназначенная для добавления нового клиента в таблицу `taxi.client`.

Процедура принимает следующие параметры:

- `p_client_name` — имя клиента;
- `p_account` — аккаунт клиента;
- `p_rating` — рейтинг клиента.

```
CREATE PROCEDURE taxi.add_client(  
    p_client_name VARCHAR,  
    p_account VARCHAR,  
    p_rating NUMERIC  
)  
LANGUAGE plpgsql  
AS $$  
BEGIN  
    INSERT INTO taxi.client (client_name, account, rating)  
    VALUES (p_client_name, p_account, p_rating);  
END;  
$$;
```

Проверка процедуры:

```
CALL taxi.add_client(  
    'Сергей Морозов',  
    'sergey_morozov',  
    4.6  
);
```

После выполнения процедуры в таблице `client` появился новый клиент *Сергей Морозов*.

Проверка результата:

```
SELECT * FROM taxi.client  
ORDER BY id_client;
```

Результат:

	id_client [PK] bigint	client_name character varying (80)	account character varying	rating numeric (2,1)
1	1	Иван Петров	ivan_petrov	4.9
2	2	Анна Смирнова	anna_smirnova	4.5
3	3	Олег Иванов	oleg_ivanov	4.2
4	4	Мария Соколова	maria_sokolova	5.0
5	5	Дмитрий Кузнецов	dmitry_kuznetsov	3.9
6	6	Сергей Морозов	sergey_morozov	4.6

2. Процедура изменения рейтинга клиента

Была создана процедура `taxi.update_client_rating`, предназначенная для изменения рейтинга клиента по его идентификатору.

Процедура принимает следующие параметры:

- `p_client_id` — идентификатор клиента;
- `p_new_rating` — новое значение рейтинга.

```
CREATE PROCEDURE taxi.update_client_rating(
    p_client_id BIGINT,
    p_new_rating NUMERIC
)
LANGUAGE plpgsql
AS $$
BEGIN
    UPDATE taxi.client
    SET rating = p_new_rating
    WHERE id_client = p_client_id;
END;
$$;
```

Проверка процедуры:

```
CALL taxi.update_client_rating(6, 4.7);
```

После выполнения процедуры рейтинг клиента Сергей Морозов был изменён с 4.6 на 4.7.

Проверка результата:

```
SELECT * FROM taxi.client
ORDER BY id_client;
```

	id_client [PK] bigint	client_name character varying (80)	account character varying	rating numeric (2,1)
1	1	Иван Петров	ivan_petrov	4.9
2	2	Анна Смирнова	anna_smirnova	4.5
3	3	Олег Иванов	oleg_ivanov	4.2
4	4	Мария Соколова	maria_sokolova	5.0
5	5	Дмитрий Кузнецов	dmitry_kuznetsov	3.9
6	6	Сергей Морозов	sergey_morozov	4.7

3. Процедура добавления заказа

Была создана процедура taxi.add_order, предназначенная для добавления нового заказа в таблицу taxi.orders.

Процедура принимает идентификаторы клиента, сотрудника, автомобиля и тарифа, а также данные о времени поездки, стоимости, оплате и рекламации.

```
CREATE PROCEDURE taxi.add_order(
    p_client_id BIGINT,
    p_employee_id BIGINT,
    p_car_id BIGINT,
    p_tariff_id BIGINT,
    p_card_number VARCHAR,
    p_order_datetime TIMESTAMP,
    p_boarding_start TIMESTAMP,
    p_passenger_boarding TIMESTAMP,
    p_planned_arrival TIMESTAMP,
    p_actual_arrival TIMESTAMP,
    p_trip_cost INTEGER,
    p_payment_method VARCHAR,
    p_cash_payment VARCHAR,
    p_payment_status VARCHAR,
    p_complaint TEXT
)
LANGUAGE plpgsql
AS $$
BEGIN
    INSERT INTO taxi.orders
    (
        client_id,
        employee_id,
        car_id,
        tariff_id,
```

```

        card_number,
        order_datatime,
        boarding_start,
        passenger_boarding,
        plannet_arrival,
        actual_arrival,
        trip_cost,
        payment_method,
        cash_payment,
        payment_status,
        complaint
    )
VALUES
(
    p_client_id,
    p_employee_id,
    p_car_id,
    p_tariff_id,
    p_card_number,
    p_order_datatime,
    p_boarding_start,
    p_passenger_boarding,
    p_plannet_arrival,
    p_actual_arrival,
    p_trip_cost,
    p_payment_method,
    p_cash_payment,
    p_payment_status,
    p_complaint
);
END;
$$;

```

Проверка процедуры:

```

CALL taxi.add_order(
    6,
    1,
    1,
    1,
    NULL,

```

```
'2026-06-18 12:00',
'2026-06-18 12:10',
'2026-06-18 12:12',
'2026-06-18 12:40',
'2026-06-18 12:38',
850,
'наличные',
'Да',
'Да',
NULL
);
```

После выполнения процедуры в таблице orders появился новый заказ.

Проверка результата:

```
SELECT * FROM taxi.orders
ORDER BY id_order;
```

	id_order [PK] bigint	client_id bigint	employee_id bigint	car_id bigint	tariff_id bigint	card_number character varying (16)	order_datetime timestamp without time zone	boarding_start timestamp without time zone	passenger_boarding timestamp without time zone	planner_start timestamp without time zone
1	1	1	1	1	1	1111222233334444	2026-06-15 09:00:00	2026-06-15 09:10:00	2026-06-15 09:12:00	2026-06-15 09:15:00
2	2	2	2	2	1	[null]	2026-06-15 10:30:00	2026-06-15 10:40:00	2026-06-15 10:42:00	2026-06-15 10:45:00
3	3	3	3	3	2	[null]	2026-06-15 12:00:00	2026-06-15 12:10:00	2026-06-15 12:15:00	2026-06-15 12:18:00
4	4	4	1	4	3	9999000011112222	2026-06-16 08:20:00	2026-06-16 08:30:00	2026-06-16 08:31:00	2026-06-16 08:35:00
5	5	5	2	5	2	[null]	2026-06-16 14:00:00	2026-06-16 14:08:00	2026-06-16 14:10:00	2026-06-16 14:15:00
6	7	6	1	1	1	[null]	2026-06-18 12:00:00	2026-06-18 12:10:00	2026-06-18 12:12:00	2026-06-18 12:15:00

planner_arrival timestamp without time zone	actual_arrival timestamp without time zone	trip_cost integer	payment_method character varying (20)	cash_payment character varying (3)	payment_status character varying (3)	complaint text
2026-06-15 09:40:00	2026-06-15 09:38:00	750	карта	Нет	Да	[null]
2026-06-15 11:05:00	2026-06-15 11:08:00	620	наличные	Да	Да	[null]
2026-06-15 12:50:00	2026-06-15 12:55:00	1100	СБП	Нет	Да	Водитель опозд...
2026-06-16 09:00:00	2026-06-16 08:58:00	1800	карта	Нет	Да	[null]
2026-06-16 14:40:00	2026-06-16 14:45:00	950	QR	Нет	Да	[null]
2026-06-18 12:40:00	2026-06-18 12:38:00	850	наличные	Да	Да	[null]

Создание триггеров

1. Триггер проверки рейтинга клиента

Была создана триггерная функция taxi.fn_check_client_rating, которая проверяет значение рейтинга клиента перед добавлением или изменением записи.

Если рейтинг меньше 0 или больше 5, операция останавливается с помощью RAISE EXCEPTION.

```
CREATE FUNCTION taxi.fn_check_client_rating()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
BEGIN
    IF NEW.rating < 0 OR NEW.rating > 5 THEN
        RAISE EXCEPTION 'Рейтинг клиента должен быть в диапазоне от 0 до 5';
    END IF;

    RETURN NEW;
END;
$$;
```

Создание триггера:

```
DROP TRIGGER trg_check_client_rating ON taxi.client;

CREATE TRIGGER trg_check_client_rating
BEFORE INSERT OR UPDATE ON taxi.client
FOR EACH ROW
EXECUTE FUNCTION taxi.fn_check_client_rating();
```

Проверка корректных данных:

```
CALL taxi.add_client(
    'Тестовый Клиент',
    'test_client',
    4.3
);
```

Проверка некорректных данных:

```
CALL taxi.add_client(
    'Ошибка Рейтинга',
```

```
'bad_rating_client',  
7.0  
);
```

В результате попытка добавить клиента с рейтингом 7.0 была заблокирована. Была получена ошибка:

```
ОШИБКА: Рейтинг клиента должен быть в диапазоне от 0 до 5  
CONTEXT: функция PL/pgSQL taxi.fn_check_client_rating(), строка 4, оператор RAISE  
SQL-оператор: "INSERT INTO taxi.client (client_name, account, rating)  
VALUES (p_client_name, p_account, p_rating)"  
функция PL/pgSQL taxi.add_client(character varying,character varying,numeric), строка 3,  
оператор SQL-оператор  
  
SQL-состояние: P0001
```

2. Триггер проверки стоимости поездки

Была создана триггерная функция taxi.fn_check_trip_cost, которая проверяет стоимость поездки перед добавлением или изменением заказа.

Если стоимость поездки меньше или равна 0, операция останавливается.

```
CREATE FUNCTION taxi.fn_check_trip_cost()  
RETURNS TRIGGER  
LANGUAGE plpgsql  
AS $$  
BEGIN  
    IF NEW.trip_cost <= 0 THEN  
        RAISE EXCEPTION 'Стоимость поездки должна быть больше 0';  
    END IF;  
  
    RETURN NEW;  
END;  
$$;
```

Создание триггера:

```
DROP TRIGGER trg_check_trip_cost ON taxi.orders;  
  
CREATE TRIGGER trg_check_trip_cost  
BEFORE INSERT OR UPDATE ON taxi.orders  
FOR EACH ROW
```

```
EXECUTE FUNCTION taxi.fn_check_trip_cost();
```

Проверка корректных данных:

```
CALL taxi.add_order(  
    6,  
    1,  
    1,  
    1,  
    NULL,  
    '2026-06-18 13:00',  
    '2026-06-18 13:10',  
    '2026-06-18 13:12',  
    '2026-06-18 13:40',  
    '2026-06-18 13:38',  
    900,  
    'наличные',  
    'Да',  
    'Да',  
    NULL  
);
```

Проверка некорректных данных:

```
CALL taxi.add_order(  
    6,  
    1,  
    1,  
    1,  
    NULL,  
    '2026-06-18 14:00',  
    '2026-06-18 14:10',  
    '2026-06-18 14:12',  
    '2026-06-18 14:40',  
    '2026-06-18 14:38',  
    0,  
    'наличные',  
    'Да',  
    'Да',  
    NULL  
);
```

В результате попытка добавить заказ со стоимостью 0 была заблокирована. Была получена ошибка:

```
ОШИБКА: Стоимость поездки должна быть больше 0
CONTEXT: функция PL/pgSQL taxi.fn_check_trip_cost(), строка 4, оператор RAISE
```

3. Триггер автоматического добавления рекламации при опоздании

Была создана триггерная функция taxi.fn_auto_complaint_late_arrival, которая автоматически добавляет рекламацию, если фактическое прибытие позже планового времени прибытия.

```
CREATE FUNCTION taxi.fn_auto_complaint_late_arrival()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
BEGIN
    IF NEW.actual_arrival > NEW.planned_arrival
        AND NEW.complaint IS NULL THEN
        NEW.complaint := 'Автоматическая рекламация: поездка завершена позже
планового времени';
    END IF;

    RETURN NEW;
END;
$$;
```

Создание триггера:

```
DROP TRIGGER trg_auto_complaint_late_arrival ON taxi.orders;

CREATE TRIGGER trg_auto_complaint_late_arrival
BEFORE INSERT OR UPDATE ON taxi.orders
FOR EACH ROW
EXECUTE FUNCTION taxi.fn_auto_complaint_late_arrival();
```

Проверка работы триггера:

```
CALL taxi.add_order(
    6,
    1,
    1,
    1,
```

```

NULL,
'2026-06-18 15:00',
'2026-06-18 15:10',
'2026-06-18 15:12',
'2026-06-18 15:40',
'2026-06-18 15:55',
950,
'наличные',
'Да',
'Да',
NULL
);

```

Проверка результата:

```

SELECT id_order, client_id, plannet_arrival, actual_arrival, complaint
FROM taxi.orders
ORDER BY id_order;

```

В результате в поле complaint автоматически появилась запись:

Автоматическая рекламация: поездка завершена позже планового времени

4. Триггер логирования изменений заказов

Для хранения информации об изменениях в таблице orders была создана таблица логов taxi.orders_log.

```

CREATE TABLE taxi.orders_log (
  id_log BIGSERIAL PRIMARY KEY,
  operation_type VARCHAR(20) NOT NULL,
  order_id BIGINT,
  log_text TEXT,
  log_time TIMESTAMP NOT NULL DEFAULT NOW()
);

```

Была создана триггерная функция taxi.fn_orders_log, которая записывает в таблицу логов информацию об операциях INSERT, UPDATE и DELETE.

```

CREATE FUNCTION taxi.fn_orders_log()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$

```

```

BEGIN
  IF TG_OP = 'INSERT' THEN
    INSERT INTO taxi.orders_log(operation_type, order_id, log_text)
    VALUES (
      'INSERT',
      NEW.id_order,
      'Добавлен новый заказ с id_order = ' || NEW.id_order
    );
    RETURN NEW;

  ELSIF TG_OP = 'UPDATE' THEN
    INSERT INTO taxi.orders_log(operation_type, order_id, log_text)
    VALUES (
      'UPDATE',
      NEW.id_order,
      'Изменён заказ с id_order = ' || NEW.id_order
    );
    RETURN NEW;

  ELSIF TG_OP = 'DELETE' THEN
    INSERT INTO taxi.orders_log(operation_type, order_id, log_text)
    VALUES (
      'DELETE',
      OLD.id_order,
      'Удалён заказ с id_order = ' || OLD.id_order
    );
    RETURN OLD;
  END IF;

  RETURN NULL;
END;
$$;

```

Создание триггера:

```

DROP TRIGGER trg_orders_log ON taxi.orders;

CREATE TRIGGER trg_orders_log
AFTER INSERT OR UPDATE OR DELETE ON taxi.orders
FOR EACH ROW
EXECUTE FUNCTION taxi.fn_orders_log();

```

Проверка операции INSERT:

```
CALL taxi.add_order(  
    6,  
    1,  
    1,  
    1,  
    NULL,  
    '2026-06-18 16:00',  
    '2026-06-18 16:10',  
    '2026-06-18 16:12',  
    '2026-06-18 16:40',  
    '2026-06-18 16:38',  
    1000,  
    'наличные',  
    'Да',  
    'Да',  
    NULL  
);
```

Проверка операции UPDATE:

```
UPDATE taxi.orders  
SET trip_cost = trip_cost + 100  
WHERE id_order = (  
    SELECT MAX(id_order)  
    FROM taxi.orders  
);
```

Проверка операции DELETE:

```
DELETE FROM taxi.orders  
WHERE id_order = (  
    SELECT MAX(id_order)  
    FROM taxi.orders  
);
```

Проверка таблицы логов:

```
SELECT * FROM taxi.orders_log  
ORDER BY id_log;
```

В результате в таблице orders_log были зафиксированы операции INSERT, UPDATE и DELETE.

	id_log [PK] bigint	operation_type character varying (20)	order_id bigint	log_text text	log_time timestamp without time zone
1	1	INSERT	12	Добавлен новый заказ с id_order = ...	2026-06-24 04:05:15.037162
2	2	UPDATE	12	Изменён заказ с id_order = 12	2026-06-24 04:05:48.408187
3	3	DELETE	12	Удалён заказ с id_order = 12	2026-06-24 04:06:22.51871

5. Триггер запрета удаления клиента с заказами

Была создана триггерная функция taxi.fn_prevent_client_delete_with_orders, которая запрещает удаление клиента, если у него есть связанные записи в таблице orders.

```
CREATE FUNCTION taxi.fn_prevent_client_delete_with_orders()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
BEGIN
    IF EXISTS (
        SELECT 1
        FROM taxi.orders
        WHERE client_id = OLD.id_client
    ) THEN
        RAISE EXCEPTION 'Нельзя удалить клиента, у которого есть заказы';
    END IF;

    RETURN OLD;
END;
$$;
```

Создание триггера:

```
DROP TRIGGER trg_prevent_client_delete_with_orders ON taxi.client;

CREATE TRIGGER trg_prevent_client_delete_with_orders
BEFORE DELETE ON taxi.client
FOR EACH ROW
EXECUTE FUNCTION taxi.fn_prevent_client_delete_with_orders();
```

Проверка некорректного удаления:

```
DELETE FROM taxi.client
WHERE id_client = 1;
```

В результате операция удаления была заблокирована, так как у клиента есть заказы. Была получена ошибка:

```
ОШИБКА: Нельзя удалить клиента, у которого есть заказы
CONTEXT: функция PL/pgSQL taxi.fn_prevent_client_delete_with_orders(), строка 8,
оператор RAISE
SQL-состояние: P0001
```

Проверка корректного удаления тестового клиента без заказов:

```
DELETE FROM taxi.client
WHERE account = 'test_client';
```

В результате тестовый клиент был успешно удалён.

```
DELETE 1
```

Проверка результата:

```
SELECT * FROM taxi.client
ORDER BY id_client;
```

	id_client [PK] bigint	client_name character varying (80)	account character varying	rating numeric (2,1)
1	1	Иван Петров	ivan_petrov	4.9
2	2	Анна Смирнова	anna_smirnova	4.5
3	3	Олег Иванов	oleg_ivanov	4.2
4	4	Мария Соколова	maria_sokolova	5.0
5	5	Дмитрий Кузнецов	dmitry_kuznetsov	3.9
6	6	Сергей Морозов	sergey_morozov	4.7

Вывод

В ходе выполнения лабораторной работы были созданы и проверены процедуры, функции и триггеры для базы данных «Служба заказа такси».

Были разработаны три процедуры: процедура добавления клиента, процедура изменения рейтинга клиента и процедура добавления заказа. Работа процедур была проверена с помощью команды CALL.

Также были созданы триггерные функции и триггеры для проверки корректности данных и автоматизации действий в базе данных. Были реализованы проверки рейтинга клиента и стоимости поездки, автоматическое добавление рекламации при опоздании, логирование операций INSERT, UPDATE и DELETE по таблице orders, а также запрет удаления клиента, если у него существуют связанные заказы.

В результате выполнения лабораторной работы были получены практические навыки создания процедур, функций и триггеров в PostgreSQL, а также проверки их работы на корректных и некорректных данных.